Titre du document : Clean Code in Modern C++	**Auteur original :** Adedamola Ajayi
Langage & Norme : C++ (C++11 à C++20)	**Sujet :** Clean Code, Architecture, RAII, STL

GUIDE DE SYNTHÈSE TECHNIQUE

Clean Code en C++ Moderne

Principes d'expression d'intention, de gestion explicite de la propriété et de conception robuste.

Le C++ moderne dépasse la simple accumulation de nouvelles fonctionnalités de langage. Il s'agit d'exploiter les capacités du C++20 pour exprimer clairement l'**intention**, garantir la **justesse** du code et gérer rigoureusement la **propriété des ressources**. Un code propre se traduit par des contrats explicites et une gestion déterministe de la mémoire.

1. Le code propre commence par l'interface

En C++, la signature d'une fonction constitue un contrat exécutoire. L'utilisation systématique de la correction vis-à-vis des constantes (`const-correctness`) et des types appropriés communique explicitement les attentes sans nécessiter de code défensif additionnel.

[LANGAGE: C++] [TYPE: CONTRAT D'INTERFACE CLEAN CODE]

```cpp
// Signature explicite : const-correctness et passage par référence constante
int sum(const std::vector<int>& values) {
    return std::accumulate(values.begin(), values.end(), 0);
}
// Explication : Garantie de non-mutation et absence de transfert de propriété
```

2. Propriété et durée de vie : Gestion RAII

Le motif **RAII** (*Resource Acquisition Is Initialization*) lie la durée de vie des ressources à leur portée (scope). Il garantit une libération déterministe de la mémoire et des descripteurs de fichiers dès la sortie du bloc d'exécution.

[LANGAGE: C++] [MOTIF: RAII / GESTION DE RESSOURCEMENT]

```cpp
class File {
public:
    explicit File(const std::string& path)
        : handle(std::fopen(path.c_str(), "r")) {
        if (!handle) {
            throw std::runtime_error("Failed to open file");
        }
    }
    ~File() {
        if (handle) { std::fclose(handle); } // Libération automatique
    }
private:
    std::FILE* handle;
};
```

3. Privilégier la sémantique de valeur

Manipuler des objets par valeur évite les états mutables partagés implicites. Cela garantit l'isolation des données et simplifie le raisonnement local et le débogage.

> **Règle d'or :** Réduire au maximum le partage d'état mutable. Utiliser le passage par valeur ou par référence constante par défaut.

4. Exploitation délibérée de la bibliothèque standard (STL)

Préférer les algorithmes standards (ex: `std::count_if`, `std::accumulate`) aux boucles explicites réinventées pour améliorer l'expressivité et supprimer les risques de bugs hors-limites.

5. Rendre l'échec explicite avec std::optional

Remplacer les valeurs sentinelles trompeuses (ex: retourner `-1`) par des types expressifs de la bibliothèque standard comme `std::optional<T>`.

[CODE INCORRECT / ANTI-PATTERN] VALEUR SENTINELLE AMBIGUË

```
int parse_int_bad(const std::string& text) {
    try { return std::stoi(text); }
    catch (...) { return -1; } // Problème : -1 peut être une valeur valide !
}
```

[CODE CORRECT / BEST PRACTICE] TYPE STD::OPTIONAL EXPLICITE

```
std::optional<int> parse_int(const std::string& text) {
    try { return std::stoi(text); }
    catch (...) { return std::nullopt; } // Échec explicite et sûr
}
```

6. Synthèse des 5 piliers du Clean Code C++

Tableau d'analyse comparative des principes C++ Moderne

AXE DE CONCEPTION	PRINCIPE CLEAN CODE APPLIQUÉ
PROPRIÉTÉ (OWNERSHIP)	Gérée exclusivement par RAII et types intelligents (smart pointers).
MUTABILITÉ	Restreinte par l'usage systématique de `const`.
DURÉE DE VIE (LIFETIME)	Déterministe et strictement liée au scope.
DESIGN D'API	Auto-documentée par des signatures de fonctions explicites.
GESTION D'ERREURS	Types explicites (ex: `std::optional`, `std::expected`).